CS2100 Midterms Cheatsheet

by @jmsandiegoo

3. Number Systems

Data Representation

Data is represented as binary (base 2)
Units:

- **Byte:** 8-bits
- **Nibble:** 4-bits
- **Word:** 1 / 2 / 4 bits etc. based on architecture

$Nbits = 2^N values$
$Mvalues = log_2 M$ **take ceiling value*

Decimal (base 10)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9$

Binary (base 2)

$Symbols = 0, 1$
** represented by prefix 0b*

Octal (base 8)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7$
** represented by prefix 0*

Hexadecimal (base 16)

$Symbols = 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$
** represented by prefix 0x*

Conversions

- Base-R to Decimal
  - multiply each digit with respective positional weight. $(digit \times R^{position}) + ...$
- Decimal to Base-R
  - **whole number:** Repeated division by R
    ** to find remainder multiply fractional value with the divisor / R*
  - **fractions:** Repeated multiplication by R Method.
    ** Until fractional product is 0 or desired number of decimal decimal places*
- Between Bases
  - partition in groups of $currR^N = convertR$

ASCII

American Standar Code for Information Interchange
7 bits plus 1 bit for odd / even parity (for error checking)
Integers (0 - 127) are interchangeable with Characters

Negative / Signed Numbers

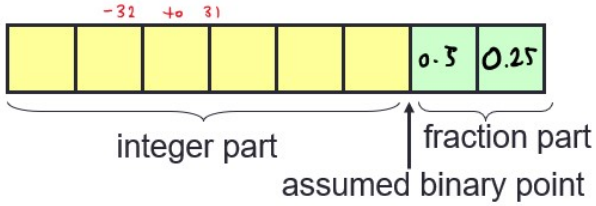
- **Sign-and-Magnitude**
  - Sign is represented by the MSB '0' is positive
  - Range for 8-bit:
    $-2^{n-1} - 1 = -127_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - Two zeroes: $00000000 = +0_{10}$ & $10000000 = -0_{10}$
- **1s Complement**
  - Negated value of x: $-x = 2^n - x - 1$ / Flip the bits
  - Range for 8-bit:
    $-2^{n-1} - 1 = -127_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - Two zeroes: $00000000 = +0_{10}$ & $11111111 = -0_{10}$
- **2s Complement**
  - Negated value of x: $-x = 2^n - x$ /
    $1s\ complement + 1$ / Copy from LSB until first '1' then invert rest till MSB
  - Range for 8-bit:
    $-2^{n-1} = -128_{10}$ *to* $2^{n-1} - 1 = +127_{10}$
  - One zero: $00000000 = +0_{10}$
- **Excess Representation**
  - Range of values distributed evenly between positive and negative values
  - Excess-M starts from $00..00 = -M$ onwards
  - To evenly distribute, use the -2^{n-1} range as M value

Addition / Subtraction 1s & 2s Complement

- **1s Complement**
  1. Perform binary Addition
  2. Carry out add 1 to result
  3. Check overflow if MSB result is opposite sign of A & B
- **2s Complement**
  1. Perform binary Addition
  2. Ignore carry out of MSB
  3. Check overflow 'carry in/out' of MSB different / like in 1s complement

Fixed-Point Representation

Number of bits allocated for whole and fractional part are fixed For example:



If 2s complement is used:
$011010.11_{2s} = 26.75_{10}$
$111110.11_{2s} = -000001.01_2 = -1.25_{10}$
Limitations: limited range and precision and immutable once implemented

Floating-Point Representation (IEE 754)

Allows to represent very large or small numbers copared to fixed point such as:
0.23×10^{23} (very large positive number)
0.5×10^{-37} (very small positive number)
-0.2397×10^{-18} (very small negative number)

IEE 754 Single precision (32-bits)

Consists of 3 main components:

- Sign (1-bit)
  - 0/1: Positive / Negative
- Exponent (8-bit w/ excess-127)
  - $exponent + 127 = 129 = 10000001_2$
- Mantissa (fraction) (23-bit)
  - nomralised with an implicit leading bit 1
  - $110.1_2 \rightarrow 1.101_2 \times 2^2$ (101 is stored in mantissa)

$-6.5_{10} = -110.1_2 = -1.101_2 \times 2^2$

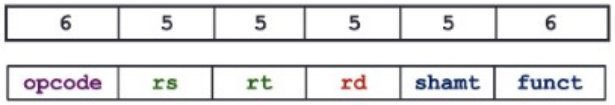
Exponent = 2 + 127 = 129 = 10000001_2

We may write the 32-bit representation in hexadecimal:

$1\ 10000001\ 10100000000000000000000_2 = C0D00000_{16}$

9. MIPS Instruction Encoding

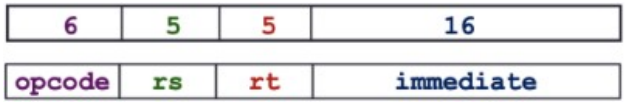
R-Format



- Opcode 000000 for R-Format
- Each field is unsigned integer
- $\$arith$: shamt is always 0, $\$shift$: rs is always 0

** add, sub, and, or, nor, slt, srl, sll, etc.*

I-Format



- Opcode is not 000000 for I-Format
- Immediate is a signed integer except bitwise operations (andi, ori, xori)

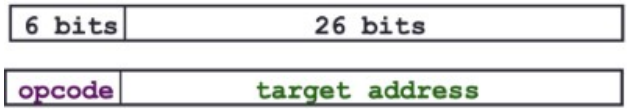
** addi, andi, ori, slti, lw, sw, beq, bne, etc.*

PC-Relative Addressing

Program Counter: special register that keeps address of the current / next instruction (but think of it as PC + 4)
Relative target address (imm): $\pm 2^{15} words = \pm 2^{17} bytes$ from the PC
** due to word aligned instructions and immediate is auto multiplied by 4*
Branch Calculation:

- **Branch Taken**: $PC = (PC + 4) + (immediate + 4)$
- **Branch Not Taken**: $PC = (PC + 4)$

J-Format



- Jump anywhere in the memory
- MIPS will take the 4 MSBs from PC + 4 and make add it infront of the target address
- 2 LSBs removed as it is always 00 due to word alignment
- **Target address**: 28 bits

11. Processor Datapath

MIPS Instruction Execution

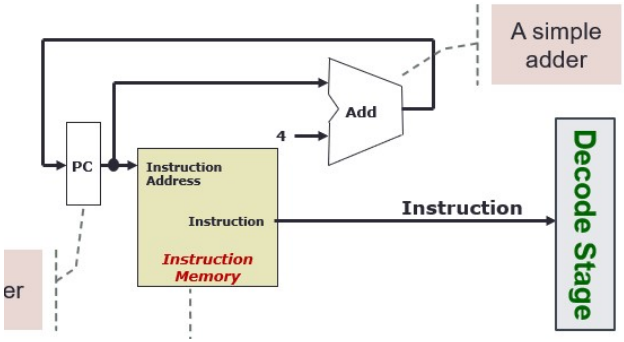
	`add $rd, $rs, $rt`	`lw $rt, ofst($rs)`	`beq $rs, $rt, ofst`
Fetch Decode	*standard*	*standard*	*standard*
Operand Fetch	○ Read [**$rs**] as *opr1* ○ Read [**$rt**] as *opr2*	○ Read [**$rs**] as *opr1* ○ Use ***ofst*** as *opr2*	○ Read [**$rs**] as *opr1* ○ Read [**$rt**] as *opr2*
Execute	*Result* = *opr1* + *opr2*	○ *MemAddr* = *opr1* + *opr2* ○ Use *MemAddr* to read from memory	*Taken* = (*opr1* == *opr2*)? *Target* = (**PC**+4) + **ofst**×4
Result Write	*Result* stored in **$rd**	*Memory* data stored in **$rt**	if (*Taken*) **PC** = *Target*

▪ **opr** = operand ▪ **MemAddr** = address ▪ **ofst** = offset

Fetch Stage

1. Fetch the instruction from memory with **Program Counter (PC)**
   ** PC is a special register*
2. Increment PC by 4 to get next address
3. Output is provided to Decode / Operand Fetch Stage using the Instruction Memory. ** address → binary instruction*

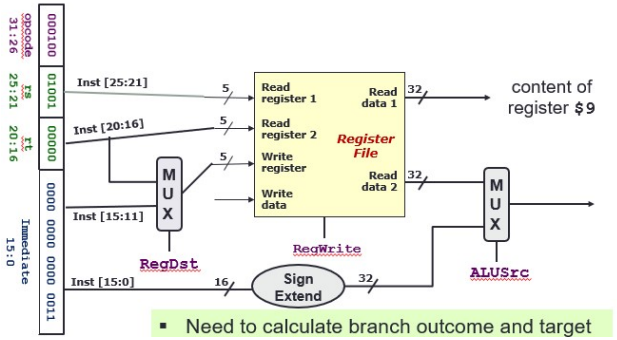
PC is read during first half of clock period and updated at next rising clock edge



Decode / Operand Fetch Stage

Gather data from instruction field:

1. Input is the binary instruction from Fetch Stage
2. Read opcode to determine instruction format and field lengths
3. Fetch data from necessary registers into the Write Register according to RegDst Signal (Rt / Rd)
4. Output is the necessary operands to be conducted in ALU Stage



Multiplexer

Selects a specific input from multiple input lines. Consists of:

- Inputs: n lines of same width (32-bits)
- Control: m bits where $n = 2^m$. Specifies which input to choose
- Output: The selected input based on i^{th} input is $control = i$

ALU Stage / Execution Stage

Performs the following operations:

- Arithmetic, Shifting & Logical
- Memory operation - address calculation
- Branch operaton - register comparison & target address calculation

1. Input is the two 32-bits numbers from Decode Stage 2nd Operand is chosed according to ALUSrc signal
2. With the help of 4-bit ALUControl signal, the respective operation is conducted
3. Outputs ALUresult & a 1-bit signal isZero

Branching in ALU

1. Identify the branch outcome through isZero to handle equal / not equal
2. Calculate Branch address with PC and Offset (Immediate value)
3. Output will be used by PCSrc signal

Memory Stage

Mainly for memory read and write instructions **lw / sw**. Other instructions remains idle in this stage and instead ALU result passed into Register Write Stage

1. Input computation result to be used as Memory Address. Also Write Data for sw (Register Data 2)
2. Read or write data on the memory address depending on MemRead/Write Signal
3. Output memory address content to Register Write stage (lw)

Register Write Stage

Routes the correct result into the Write Data in register file. Instructions like stores, branches and jumps remains idle in this stage.

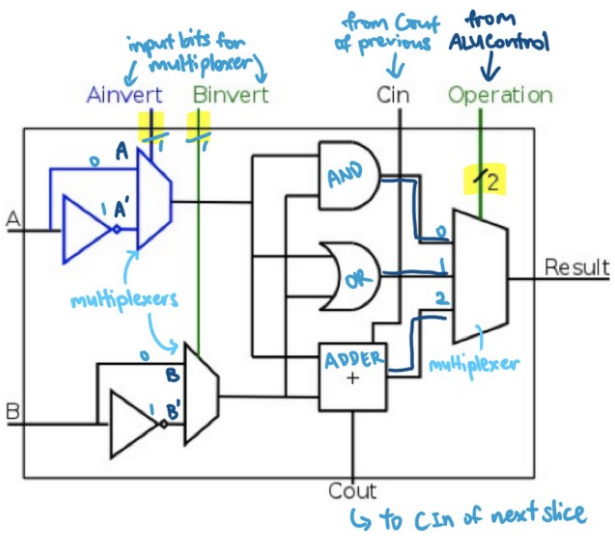
1. Input computation result from Memory or ALU, which is specified by MemToReg signal
2. Outputs the selected result into the Write Register in the register file

12. Processor Control

Control Signals

- **RegDst** - (@Decode/OperandFetch) Selects the destination of WriteRegister in register file
  - 0/1: WriteRegister = Ins[20:16] (rt) / Inst[15:11] (rd / immediate)
- **RegWrite** - (@Decode/OperandFetch) Indicates if writing on WriteRegister is enabled
  - 0/1: No register write / WD write to WR
- **ALUSrc** - (@ALU) Determines the 2nd ALU input
  - 0/1: Read Data 2 / SignExtend(Inst15:0) (Sign extended Immediate)
- **ALUcontrol** - (@ALU) Determines the type of operation to execute
  - 0000: AND
  - 0001: OR
  - 0010: add
  - 0110: subtract
  - 0111: slt
  - 1100: NOR
- **MemRead** - (@Memory) Indicates if read from memory using Address returned by ALU
  - 0/1: No read / Reads memory with Address returned by ALU
- **MemWrite** - (@Memory) Indicates if write to memory using RD2 of Register File
  - 0/1: No memory write / Writes RD2 into the memory in Address returned by ALU
- **MemToReg** - (@RegWrite) Indicates which one to write back into WriteData in register file
  - 0/1: WriteData = ALU result / Memory read data
- **PCSrc** - (@ALU) Indicates whether to take branch address or not
  - 0/1: $PC = PC + 4$ / $PC = SignExt(Inst[15:0]) << 2 + (PC + 4)$
  - PCSrc = isBranch AND isZero

ALU



ALU operation controlled by 2-bit ALUControl

** Note: operation is determined by the last 2-bits **
Subtraction has Cin = 1 for bit-0

Multilevel Decoding

Used to generate the ALUcontrol signal. *(Simpler and more efficient design than Brute Force approach)*

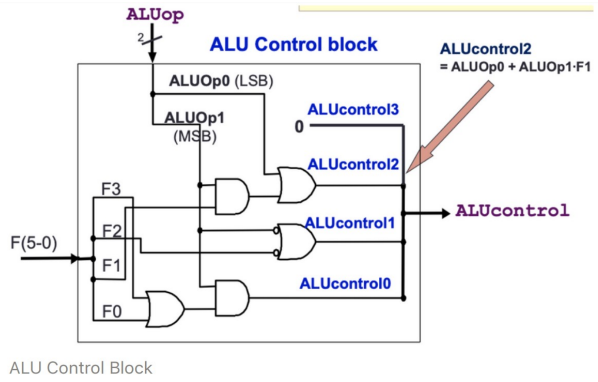
1. Generate a 2-bit ALUop signal from opcode
2. Use ALUop and Function code (for R-type) to generate 4-bit ALUcontrol signal

Opcode	ALUop	Instruction Operation	Funct field	ALU action	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
R-type	10	subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	set on less than	101010	set on less than	0111

Generation of 2-bit ALUop signal will be discussed later

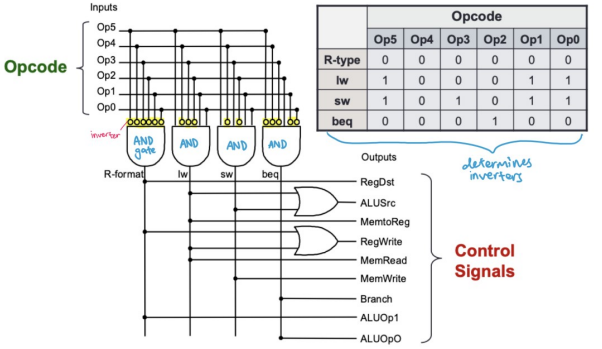
Instruction Type	ALUop
lw / sw	00
beq	01
R-type	10

ALUcontrol	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	slt
1100	NOR



ALU Control Block

Control Combinational Circuit



	Opcode					
	Op5	Op4	Op3	Op2	Op1	Op0
R-type	0	0	0	0	0	0
lw	1	0	0	0	1	1
sw	1	0	1	0	1	1
beq	0	0	0	1	0	0

Instruction Execution

How the implementation of fetch, decode, memory, write, etc. are actually done
Mainly three things:

1. Read contents from register / memory
2. Perform some form of computation
3. Write results to register / memory

Single Cycle Execution

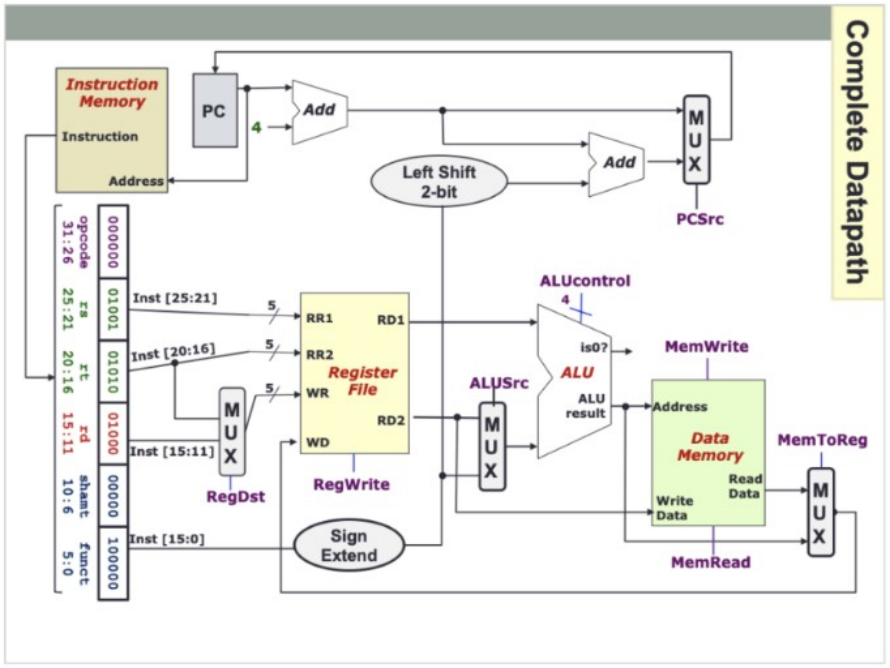
All performed within a clock period.
disadvantage: all instructions will take the same time as the slowest one.

Multi Cycle Execution

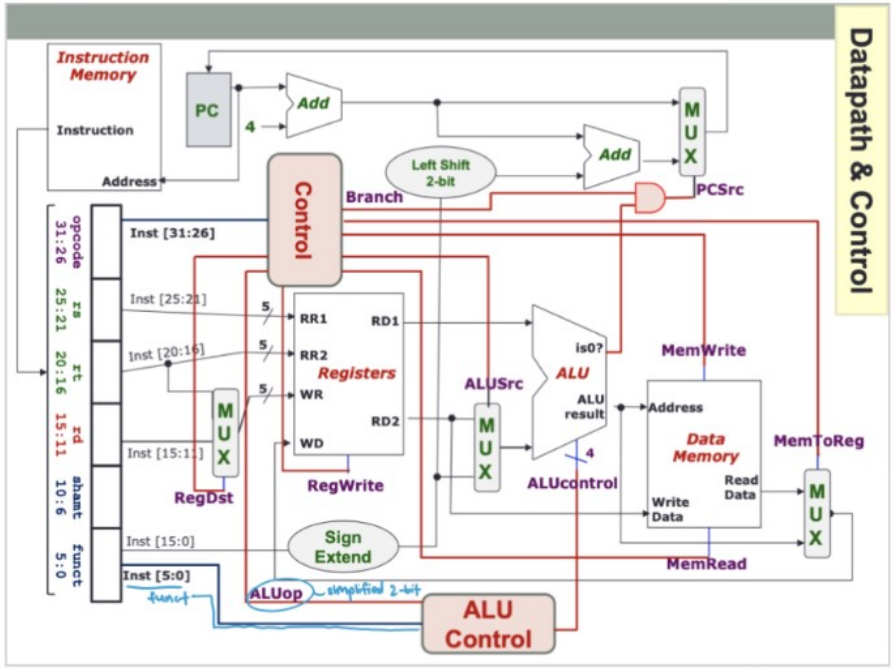
Break up instructions into execution steps (Fetch, Decode, ALU Operation, Mem read/write, Register Write). Each execution is one clock cycle. **disadvantage:** time taken depends on instructions and may not necessarily be faster.

Datapath & Control Diagrams

▼ datapath



▼ datapath & control



	RegDst	ALUSrc	MemTo Reg	Reg Write	Mem Read	Mem Write	Branch	ALUop	
								op1	op0
R-type	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X (writing to address (NOT register))	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1